BOARDWISE

TESTING DOCUMENT POLICY

Works On My Machine™

Version: 2.0

INTRODUCTION

Boardwise is a community-driven platform for boardgame enthusiasts to manage collections, join communities, organise events, access a shared rulebook library (with RAG-based Q&A), and browse/trade through a peer-to-peer marketplace. This document sets out how testing is run on Boardwise; what gets tested, with what tools, under whose ownership, and what has to pass before it ships. The goal is a consistent process across the team: every feature is planned, tested, documented, and reviewed the same way, so the system stays aligned with the SRS and holds up as reliable, usable, and secure across collections, communities, events, the rulebook library (with RAG-based Q&A), and the marketplace.

What follows covers our testing objectives, the testing types used at each layer (unit, integration, component, E2E), the tools and environments behind them, how defects get logged and closed out, the acceptance criteria a feature has to clear before merge, and who's responsible for what across the testing lifecycle.

Testing Objectives

- Verify that all functional requirements defined in the SRS are correctly implemented.
- Ensure frontend-backend integration works as specified by each services's API contract
- Detect defects early, before merge into shared branches (frontend-dev & backend-dev).
- Validate data integrity across MongoDB Atlas persistence layer.
- Confirm JWT authentication is enforced correctly on protected endpoints.

Testing Types

Unit Testing

Individual functions, methods, or components tested in isolation using the **A**rrange - **A**ct - **A**ssert pattern

ARRANGE

Create the entities/inputs needed for the test

ACT

Call the function/method/component under test

ASSERT

Create the entities/inputs needed for the test

Springboot Example - ListingServiceTest (JUnit 5 + Mockito)

Testing ListingService.createListing() in isolation by mocking all dependencies (repositories, JWT service, S3 client) so no real database or storage calls are made.

1. Arrange:
   1.1. Set up a fake user, a mock image file, and a listing request; stub the mocked repositories to return known values
2. Act:
   2.1. Call listingService.createListing()
3. Assert
   3.1. Check the returned listing has the correct title, price, condition, and genes
   3.2. Confirm repository methods were called the expected number of times

```
@Test
void shouldCreateSaleListingWithBoardGameInRepo() {
   // ARRANGE
   when(jwtService.extractUserId(fakeToken)).thenReturn(new ObjectId());
   when(listingRepository.save(any(Listing.class))).thenReturn(fakeSavedListing);
   when(boardGameRepository.findByTitle("Ludo")).thenReturn(Optional.of(bg));

   // ACT
   ListingResponse res = listingService.createListing(listingRequest, fakeToken,
mockImage);

   // ASSERT
   assertEquals("Ludo", res.gameTitle());
   assertEquals(250, res.price());
   verify(listingRepository, times(2)).save(any(Listing.class));
}
```

Python Example - text_extractor.py (pytest + mock)

Testing extract_txt(), which reads text from an uploaded rulebook PDF by attempting to extract the text using OCR as a fallback. External libraries (fitz, pytesseract) are mocked so no real pdf or OCR processing happens during the test

1. Arrange:
   1.1. Mock a PDF page and set what text it should “contain”
2. Act:
   2.1. Call extract_text()
3. Assert

3.1. Check the correct text is returned and that the OCR only runs when its needed

```
@patch("app.services.extractor.fitz.open")
def test_extract_text_accepts_native_pdf_text(mock_fitz_open, safe_pdf_bytes):
    # ARRANGE
    mock_page = MagicMock()
    mock_page.get_text.return_value = "some long readable text..."
    mock_fitz_open.return_value.__enter__.return_value.__iter__.return_value =
[mock_page]

    # ACT
    success, text = extract_text(safe_pdf_bytes)

    # ASSERT
    assert success is True
    mock_page.get_pixmap.assert_not_called()  # OCR was not needed
```

Vitest Service Testing Example (Vitest + @nuxt/test-utils)

```
import { describe, it, expect, vi, beforeEach } from 'vitest'
import { useCreateListing } from '~/composables/useCreateListing'

// Mock the module the composable depends on
vi.mock('~/services/listingApi', () => ({
  createListingRequest: vi.fn(),
}))

import { createListingRequest } from '~/services/listingApi'

describe('useCreateListing', () => {
  beforeEach(() => {
    vi.clearAllMocks()
  })

  it('should create a listing and return the mapped response', async () => {
    // ARRANGE
    const fakePayload = {
      gameTitle: 'Ludo',
      price: 250,
      condition: 'GOOD',
    }

    const fakeApiResponse = {
      id: 'abc123',
      gameTitle: 'Ludo',
```

```
    price: 250,
    condition: 'GOOD',
  }

  vi.mocked(createListingRequest).mockResolvedValue(fakeApiResponse)

  const { createListing } = useCreateListing()

  // ACT
  const result = await createListing(fakePayload)

  // ASSERT
  expect(result.gameTitle).toBe('Ludo')
  expect(result.price).toBe(250)
  expect(result.condition).toBe('GOOD')
  expect(createListingRequest).toHaveBeenCalledTimes(1)
  expect(createListingRequest).toHaveBeenCalledWith(fakePayload)
 })
})
```

Integration Testing

Integration tests cover:

- Auth: JWT validation and role/ownership on protected endpoints
- Rulebook pipeline: upload to the R2 storage to MongoDB metadata persistence layer to retrieval
- Board games and community listings: service to repository to Mongodb ATLAS verifying that the persisted document matches the request payload
- Event lifecycle: Event creation to Attendee/RSVP sync

Component Testing

Vue 3 component behaviour tested in isolation using Vitest and nuxt's test utilities (@nuxt/test-utils)
Coverage areas:

Props: correct rendering given various prop combinations
Emits: correct events fired on user interaction (e.g @submit, @rsvp)
Rendering: conditional rendering, loading/error/empty states
Composables: isolated logic tested Independently of components where possible

```
import { describe, it, expect } from 'vitest'
import { mount } from '@vue/test-utils'
```

```
import RsvpButton from '~/components/events/RsvpButton.vue'

describe('RsvpButton', () => {
  it('renders "RSVP" when the user has not joined the event', () => {
    const wrapper = mount(RsvpButton, {
      props: { eventId: 'evt-1', isAttending: false, loading: false }
    })
    expect(wrapper.text()).toContain('RSVP')
    expect(wrapper.find('[data-test="spinner"]').exists()).toBe(false)
  })

  it('shows a loading state while the RSVP request is in flight', () => {
    const wrapper = mount(RsvpButton, {
      props: { eventId: 'evt-1', isAttending: false, loading: true }
    })
    expect(wrapper.find('[data-test="spinner"]').exists()).toBe(true)
  })

  it('emits @rsvp with the event id when clicked', async () => {
    const wrapper = mount(RsvpButton, {
      props: { eventId: 'evt-1', isAttending: false, loading: false }
    })
    await wrapper.find('button').trigger('click')
    expect(wrapper.emitted('rsvp')).toBeTruthy()
    expect(wrapper.emitted('rsvp')[0]).toEqual(['evt-1'])
  })
})
```

End-to-End (E2E) Testing

Example in Playwright:

```
test('Navigate From Landing page to Rulebooks and view a rulebook',async ({page})=>{
  const targetPage = homepage ;
  const sideBar = page.locator('nav.v-navigation-drawer--right');

  // land on page
  await page.goto(targetPage);

  // click first rulebook button
  const rulebooksBtn = page.getByRole('button', { name: 'Rulebooks' }).first(); //top rulebook
button
  await rulebooksBtn.click();
  await expect(page).toHaveURL(/.*\/library/);
  await expect(page.getByRole('heading', { name: /library/i })).toBeVisible();
```

```
  //check to see if rulebook exists
  const rulebookCards =  page.locator('.d-flex.overflow-x-auto');

  //wait for cards to actually show up
  await rulebookCards.waitFor({state: 'visible'});

  // find all the listing cards
  const cards = rulebookCards.locator('.v-card');

  //VERY FRAGILE BANDILEEEEE!!!
  //wait to load
  await expect(cards.first()).toBeVisible({ timeout: 10000 }); // loading

  //cards
  const firstRulebookCard = cards.first();

  const CardTitle = await firstRulebookCard.locator('.text-body-2').first().textContent();

  await firstRulebookCard.click();


  //check if sidebar opened
  await expect(sideBar).toBeVisible({timeout: 5000});
  await expect(sideBar).toHaveClass(/v-navigation-drawer--active/);

  const rulebookTitle = await sideBar.locator('h2').textContent();

  //check if loaded data matches for integrity purposes
  await expect(CardTitle.trim()).toMatch(rulebookTitle.trim());

  //GO TO RULEBOOK PAGE
  const readRulebookButton = page
    .locator('nav.v-navigation-drawer--right')
    .getByRole('button', { name: 'Read Rulebook' });

  await readRulebookButton.click();

  await expect(page).toHaveURL(/\/library\/read\/.+/);


  //ensure the right rulebook is seen
  const readingPageTitle = await page.locator('h1').textContent();
  await expect(readingPageTitle.trim()).toMatch(CardTitle.trim());

});
```

Testing Tools & Environments

Backend (Spring Boot/ Java)

Tool	Purpose
JUnit 5	Test runner/framework: @Test @BeforeEach Assertions: assertEquals, assertThat, assertThrows, assertNotThrows
Mockito	Mocking framework for unit tests @Mock when(...)thenReturn(...), verify(...)
Spring Boot Test (@SpringBootTest)	Boots the full application for integration tests
MockMvc	Simulates HTTP requests against controllers without starting a real server
AssertJ	Assertion Library often paired with JUnit for readability (assertThat(x).hasSize(1))

Backend (Python/ FastAPI)

Tool	Purpose
pytest	Test runner: Fixtures, parameterization, assert statements
unitTest-mock	Mocking framework for unit tests @Mock when(...)thenReturn(...), verify(...)
Pytest fixture	Reusable setup (e.g. safe_pdf_bytes, test_db, test_client)

Frontend(Vue 3/ Nuxt 4)

Tool	Purpose
@nuxt/test-utils	Test runner: Fixtures, parameterization, assert statements
@vue/test-utils	Mocking framework for unit tests @Mock when(...)thenReturn(...), verify(...)
Playwright	E2E integration testing
happy-dom	Simulated DOMO environment Vitest runs component tests in.

Backend (Cross-cutting / CI concerns)

Tool	Purpose
Docker Compose	Local usage of mongo, backend,frontend
GitHub Actions	Runs the full suite on Pull Request

Defect Management Process

1. Logging
    - Defects are reported to the relevant domain developer who logs the issues on GitHub and provides the bug tag on issue and records a severity (URGENT | HIGH | MED |LOW) and an estimated amount of effort required to solve the bug.
2. Triage
    - The team is made aware and the domain owner focuses on the defect depending on severity.
3. Fix & Verify
    - Owner fixes on a feature fix branch.
    - Fix is verified by domain owner recreating “failing action” as a test and merge if test passes.
4. Closure
    - Issue is closed only after the associated test is added/ updated to cover the defect.
5. Escalation
    - Critical defects found within 3 days of a demo are flagged to the team lead and a team decision is made on whether to fix the defect or hide the failing feature.

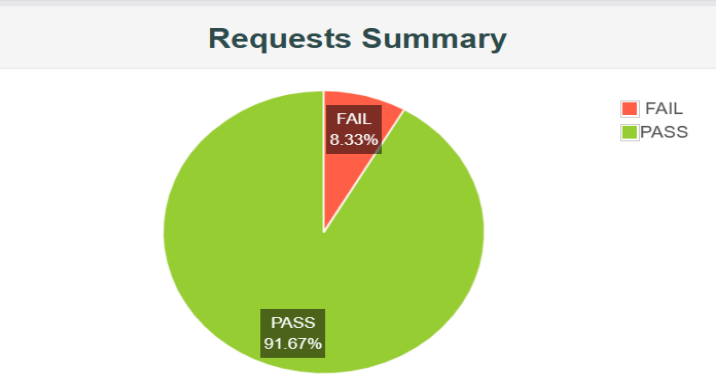
Non-Functional Requirements Testing Tools

NFR	TOOL	Purpose
Performance & Scalability	Apache JMeter	Load/stress testing, simulate concurrent users,measure response time under increased load
Availability	HetrixTools	Uptime monitoring of the deployed system, response time over time
Security	TruffleHog	Scans repository history/commits for leaked secrets and credentials.
Maintainability	SonarCloud	Static analysis: code smells, duplication, maintainability rating.

PERFORMANCE & SCALABILITY

Load testing was performed using Apache JMeter, simulating concurrent users across core endpoints. Of 120 requests executed, 91.67% passed with an average response time of 341ms. The login endpoint was the heaviest at 657ms average (max 1025ms), which is acceptable given it's a one-time cost per session. One endpoint, `GET /api/vault/rulebooks`, failed 100% of its test runs and requires investigation before the next test cycle.

Requests	Executions			Response Times (ms)							Thro
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transac
Total	120	10	8.33%	341.37	8	1025	303.50	791.00	950.40	1024.37	12.82
POST /api/auth/login	60	0	0.00%	657.43	454	1025	616.50	947.80	1004.85	1025.00	6.42
GET /api/vault/rulebooks	10	10	100.00%	20.70	11	64	15.00	60.50	64.00	64.00	1.19
GET /api/users/friends	10	0	0.00%	13.80	8	37	11.00	34.70	37.00	37.00	1.19
GET /api/users/	10	0	0.00%	24.00	13	66	19.50	62.00	66.00	66.00	1.21
GET /api/marketplace/listings/personalised	10	0	0.00%	26.90	13	92	18.00	86.30	92.00	92.00	1.20
GET /api/marketplace/listings	10	0	0.00%	19.50	11	47	15.50	44.60	47.00	47.00	1.21
GET /api/community/	10	0	0.00%	46.90	29	153	33.50	142.40	153.00	153.00	1.19



AVAILABILITY

Uptime is monitored via HetrixTools on the deployed system. Over the August 2026 period, the system recorded 94.94% uptime with one outage totalling 1 day 13 hours 40 minutes. Given free-tier hosting constraints, this is within acceptable range for a capstone deployment, though the cause of the outage should be logged and addressed to reduce

recurrence.

HetrixTools

Monthly Uptime Monitoring Report
August 2026

Monitor	Outages	Downtime	Uptime
Boardwise Website Monitor	1 (3)	1d 13 hr 40 min (1w 1d 4 hr 1 min)	94.9373% (73.6537%)

**Note: Brackets indicate values including maintenance downtimes.*

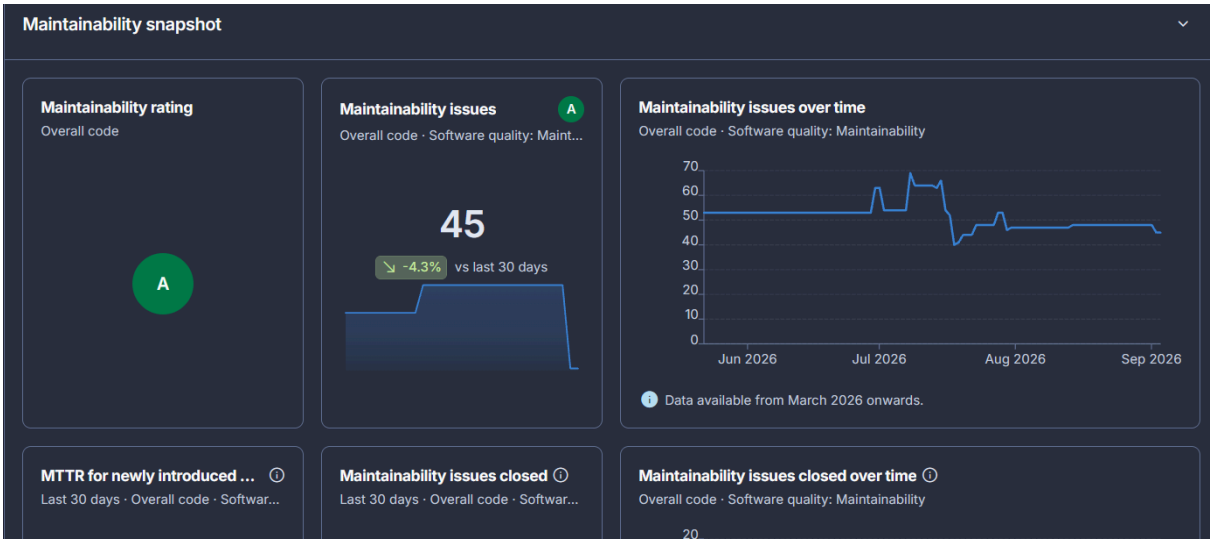
Learn how to configure Daily/Weekly/Monthly Uptime Monitor Reports for your Status Pages.

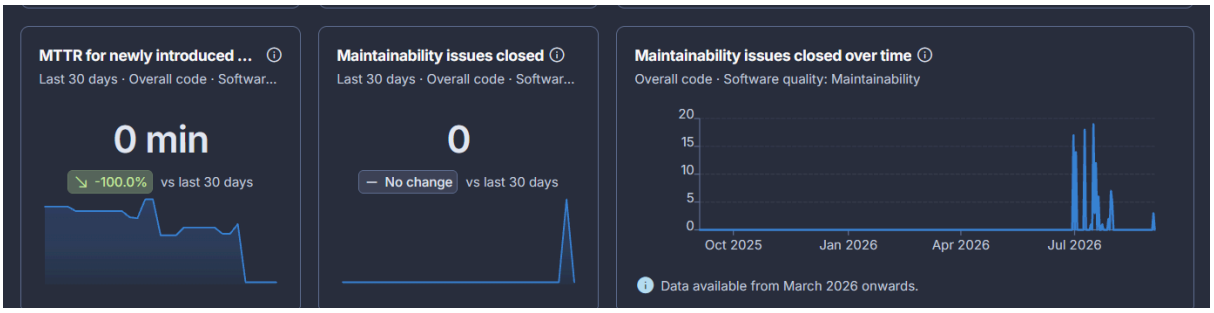
Home • Log In • Documentation

Unsubscribe from monthly Uptime Monitoring reports

MAINTAINABILITY

Code quality is tracked with SonarCloud, giving an overall maintainability rating of A. There are currently 45 open maintainability issues (mostly code smells and duplication), with 0 closed in the last 30 days. This indicates the codebase is stable but issue backlog needs active triage going forward.





Acceptance Criteria

- All acceptance criteria defined per user story are met.
- No critical or high-severity defects remain open against the feature.
- Code has been reviewed and approved by at least 2 other team members.
- The feature has been merged into the appropriate dev branch with no unresolved merge conflicts.
- Unit, Integration (for core features) E2E tests exist and pass in CI
- Previously demoed features remain fully functional (regression check passed)

Roles & Responsibilities

- Each team member is responsible for testing the features/components they develop
- All code changes go through peer review before merging into shared branches
- At least 2 of the 5 team members must review and approve a change before a merge
- Single line/ marginal changes that have no potential to break the system can be directly committed into the dev branch
- Only the team lead may PR into main
- The responsibility of testing is distributed amongst all developers, with peer review as the primary quality